ROB 498/599 · 3D Robot Perception · Fall 2026

L17 4D and occupancy forecasting

Occupancy as a learned function, as a semantic grid built from cameras, and as a forecast; how points move; how moving scenes are rebuilt; and where driving and manipulation part ways.

Wed Nov 4, 2026 • ANNOTATED build

4D and occupancy forecasting

L17 · Unit 3

Monday's world models predicted a latent vector. A planner that has to stay out of the way of a cyclist wants something it can check a path against: which parts of space will be full, and when.

Today: **occupancy** as a function, as a grid built from cameras, and as a forecast; then how points **move**.

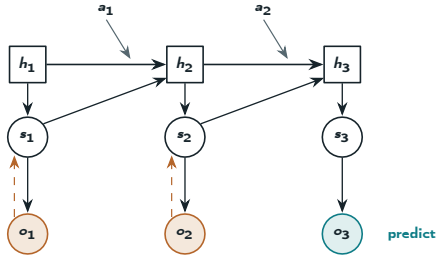
Today

1. **Finishing L16:** the RSSM's loss, PlaNet, reinforcement learning, Dreamer and DreamerV3, then I-JEPA.
2. **Occupancy Networks** (the response paper): a shape as a function, trained on sampled points, meshed with MISE.
3. **Occupancy from cameras:** TPVFormer, SurroundOcc and SparseOcc, scored with mIoU and RayIoU.
4. **Forecasting occupancy:** rendering future lidar, OccWorld's tokens, and UnO.
5. **Scene flow:** how each point moves.
6. **Dynamic scenes:** a canonical shape and a deformation field.
7. **Driving and manipulation diverge:** who makes the world move.

Each part starts with the tools it needs, built from small examples.

A latent dynamics model

where L16 stopped: both paths (RSSM)



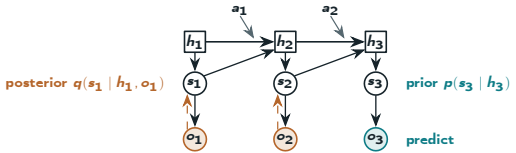
orange: observed frames and what is inferred from them; teal: predicted

$$h_t = f(h_{t-1},\ s_{t-1},\ a_{t-1}), \qquad s_t \sim p(s_t \mid h_t), \qquad o_t \sim p(o_t \mid h_t, s_t), \quad r_t \sim p(r_t \mid h_t, s_t)$$

The **recurrent state-space model**: h_t (a GRU, 200 units) carries memory; s_t (30-D diagonal Gaussian) carries uncertainty.

A latent dynamics model

4 of 4: prior, posterior, loss



orange: observed frames and what is inferred from them; teal: predicted

- **Prior** $p(s_t \mid h_t)$: the prediction before seeing o_t. Used when imagining.
- **Posterior** $q(s_t \mid h_t, o_t)$: after seeing it. Used on real frames.
- **Analogy:** a navigator's dead reckoning (prior), corrected by a sighted landmark (posterior).

$$\mathcal{L} = \sum_t \Big(\underbrace{-\ln p(o_t \mid h_t, s_t) - \ln p(r_t \mid h_t, s_t)}_{\text{reconstruct}} + \underbrace{\mathrm{KL}\big(q(s_t \mid h_t, o_t) \,\|\, p(s_t \mid h_t)\big)}_{\text{predict what you will see}} \Big)$$

PlaNet: plan in the latent space

Hafner et al., ICML 2019

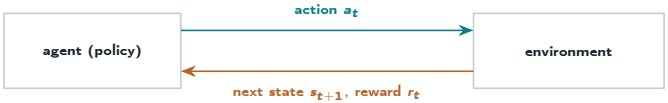
1. Filter the real frames into (h_t, s_t) with the posterior.
2. CEM over 12 actions, scored by summed predicted reward, rolled out with the **prior** only. Run the first action.
3. Every 100 updates, collect one more episode and add it to the data.



Mean return over six control tasks (Table 1), with a hundredth of the episodes. The paper also proposes **latent overshooting** (KL terms on multi-step predictions); its appendix says the final agent did not need it.

Reinforcement learning

1 of 8: learning from a score



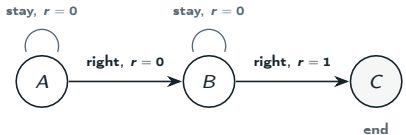
teal: what the agent chooses; orange: what the world returns

- In supervised learning (L05) every input comes with the right answer. Here the robot acts, and the world returns only a **reward**: a score, often long after the action that earned it.
- The robot has to work out for itself which of its actions led to the score. This is the **credit assignment** problem.

Analogy: learning a video game with no manual. You press buttons; the only feedback is the score.

Reinforcement learning

2 of 8: the pieces of the problem



- **State** s: what the robot knows now. Here A, B or C.
- **Action** a: what it can do. Here *right* or *stay*.
- **Transitions** $p(s' \mid s, a)$: how the world responds. This is exactly a **world model**.
- **Reward** $r(s, a)$: here 1 for reaching C, otherwise 0.

On a robot: the state is joint angles and an image, an action a motor command, the reward 1 when the block is in the bin.

Reinforcement learning

3 of 8: the return, and why discount

$$G_t = r_t + \gamma\, r_{t+1} + \gamma^2 r_{t+2} + \cdots = r_t + \gamma\, G_{t+1}$$

- The **return** adds up all the rewards still to come. The agent tries to make it large.
- The **discount** $\gamma < 1$ counts a reward k steps away as γ^k: sooner is worth more, and the sum stays finite.

weight γ^k	$k = 1$	$k = 10$	$k = 50$	$k = 100$
$\gamma = 0.9$	0.900	0.349	0.005	2.7×10^{-5}
$\gamma = 0.99$	0.990	0.904	0.605	0.366

On the chain, rewards $0, 0, 1$ give $G_0 = 0.9^2 = 0.81$. γ sets how far ahead the agent cares: about $1/(1-\gamma)$ steps.

Reinforcement learning

4 of 8: a policy, and why it should sometimes explore

$$\pi(a \mid s): \qquad \pi(\text{right} \mid A) = 0.8, \quad \pi(\text{stay} \mid A) = 0.2$$

- A **policy** says what to do in each state, as a probability for each action. In deep RL it is a network, the **actor**.
- **Exploration:** an agent that always takes its current best action never finds out whether another one is better.

Analogy: always ordering your favourite dish, or sometimes trying a new one. Trying costs a little tonight and can pay off on every later visit.

Reinforcement learning

5 of 8: value, the return you expect

$$v_\pi(s) = \mathbb{E}_\pi\big[\, G_t \mid s_t = s \,\big]$$

- The **value** of a state is the return to expect from there when following π, averaged over everything that could happen.
- It is a prediction about the future, made now. It lets the agent judge a state without playing it out.
- In deep RL it is a network, the **critic**.

Example: from s_0, two equally likely futures with returns 0.81 and 0, so $v(s_0) = 0.405$.

Analogy: a chess player who looks at a board and says *white is winning*, without playing the game to the end.

Reinforcement learning

6 of 8: learning values from experience (TD)

$$v(s) \leftarrow v(s) + \alpha \left(\underbrace{r + \gamma\, v(s')}_{\text{target}} - v(s) \right)$$

- Usually we cannot compute values exactly: the transitions are unknown and the states too many.
- So after each real step, nudge $v(s)$ towards one reward plus the current guess for the next state: **temporal-difference (TD) learning**. A guess improved by another guess is called **bootstrapping**.

$\alpha = 0.5$	**start**	**after 1**	**after 2**	**true value**
$v(A)$	0	0	0.225	0.9
$v(B)$	0	0.5	0.75	1

Reinforcement learning

- **Critic** $v(s)$: learns values by TD.
- **Actor** $\pi(a \mid s)$: after each action, asks the critic whether it went better than expected.

$$\delta = r + \gamma\, v(s') - v(s) \qquad \delta > 0: \text{ make } a \text{ more likely}; \quad \delta < 0: \text{ less likely}$$

From B, with $v(B) = 0.75$: *right* gives $\delta = 1 - 0.75 = +0.25$, so raise it; *stay* gives $\delta = 0.9 \cdot 0.75 - 0.75 = -0.075$, so lower it.

Analogy: a coach who predicts how well each play will go, and a player who repeats the plays that beat the prediction.

Reinforcement learning

8 of 8: where the experience comes from

	Who produces the steps (s, a, r, s')
model-free RL	the real robot or simulator: millions of steps, slow, and wear on the hardware
model-based RL	a learned **world model**: cheap, thousands in parallel on a GPU, but carrying the model's errors

PlaNet used its model to **plan** at every step. Dreamer uses it to **practise**: an actor-critic trained on experience the world model imagines.

Dreamer: learn the behaviour in imagination

Hafner et al., ICLR 2020; figure from DreamerV3

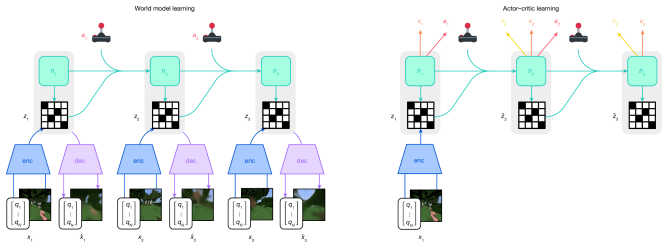


Figure: DreamerV3 (Hafner et al.), Nature 2025, Fig. 1, CC BY 4.0

- **Left, world model learning** on real frames x_t: encoder, recurrent state h_t, latent z_t, decoder. The RSSM from the start of today.
- **Right, actor-critic learning** in imagination: from one real frame the model rolls forward to $\hat{z}_2, \hat{z}_3$ and predicts actions a, rewards r and values v. No image is decoded.

Dreamer's actor and critic

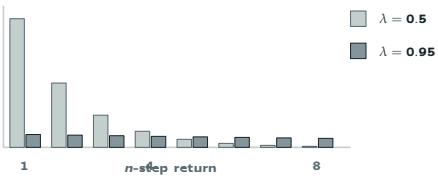
- **Critic** $v(s)$: regresses onto targets computed from imagined rewards and its own values.
- **Actor** $\pi(a \mid s)$: trained to make those targets large.
- The critic lets a 15-step imagined rollout count rewards long after it ends: stop, and add $\gamma^H v(s_{t+H})$.

Dreamer v1: 20 visual control tasks, average 823 after 5×10^6 steps, against PlaNet's 332 and model-free D4PG's 786 after 10^8.

λ-returns

what the critic is trained on

An n-step return uses n imagined rewards, then the critic: $R_t^{(n)} = r_t + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n v(s_{t+n})$. TD used $n = 1$.



- Small n: leans on the critic, which may be wrong (**bias**).
- Large n: leans on imagined rewards, which compound errors (**variance**).
- The λ-return weights $R^{(n)}$ by $(1-\lambda)\lambda^{n-1}$.

$$R_t^\lambda = r_t + \gamma\Big((1-\lambda)\, v(s_{t+1}) + \lambda\, R_{t+1}^\lambda\Big), \qquad R_H^\lambda = v(s_H)$$

Analogy: estimating a drive's arrival time. Ask the sat-nav now, or drive some legs and then ask it; the λ-return averages all of these.

DreamerV3 as a reinforcement learner

each RL piece, and where it lives

RL piece	In DreamerV3
state s	the model state (h_t, z_t): recurrent memory plus a discrete latent
transitions $p(s' \mid s, a)$	the sequence model and dynamics predictor: the RSSM's prior
reward r	a **reward head** on the model state
episode end	a **continue head**: will the episode go on? It scales the discount
value $v(s)$	the **critic**, trained on λ-returns
policy $\pi(a \mid s)$	the **actor**, trained on advantages
experience	imagined 15-step rollouts from replayed real states

Every RL quantity is computed from the model state. The actor and critic never see an image.

DreamerV3's training loop

three learners, one replay buffer

1. **Act:** run the actor on the real environment; store frames, actions, rewards and episode ends in a **replay buffer**.
2. **Learn the model:** on replayed sequences, reconstruct frames, predict rewards and ends, and train the prior towards the posterior (the KL).
3. **Imagine:** from replayed states, roll forward 15 steps with the actor and the prior. The heads score every step.
4. **Learn to act:** the critic regresses onto λ-returns; the actor raises actions whose return beat the critic.

$$\text{actor: increase } \pi(a_t \mid s_t) \text{ in proportion to } \frac{R_t^\lambda - v(s_t)}{\max(1,\ S)} \ + \text{ an entropy bonus}$$

$R_t^\lambda - v(s_t)$ is the actor-critic δ with a better target. S is the spread of recent returns, so one setting fits any reward scale.

Why one configuration is hard

four fixes, in one table

DreamerV3 uses one set of hyperparameters on over 150 tasks in 8 domains. Rewards, returns and observations differ by orders of magnitude between domains, and so does every loss term; earlier Dreamers retuned for each. DreamerV3 adds four fixes:

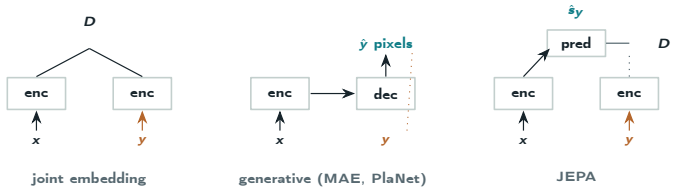
Fix	**What it handles**
symlog, two-hot	targets of unknown size: a reward of 0.01 and one of 10,000
free bits, KL balancing	the world model's KL swamping, or vanishing against, reconstruction
return normalisation	the actor's entropy bonus mattering too much or too little
1% unimix	categorical distributions going deterministic

Self-supervised learning

- **Supervised** (L05): every input comes with a human label.
- **Self-supervised:** hide part of the data and predict it from the rest. The labels come from the data itself.
- Examples so far: the autoencoder predicts its own input; the world model predicts the next frame.

The goal is usually the **representation**: an encoder whose features are useful for other tasks, learned from unlabelled images and video.

Why predict pixels at all?

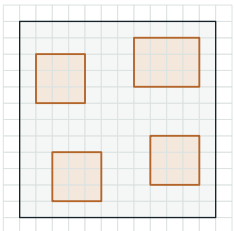


orange: the target y (real data); teal: the model's prediction; D: the distance compared

- PlaNet and Dreamer learn their state by **reconstructing frames**. Most of a frame is irrelevant to control: leaves, carpet texture, glare.
- A pixel loss spends capacity on all of it, and punishes a sharp but slightly wrong future more than a blurry average.
- **Joint-embedding predictive architecture (JEPA):** encode both, and predict the target's *code* from the context's.

I-JEPA

1 of 2: what to hide (Assran et al., CVPR 2023)

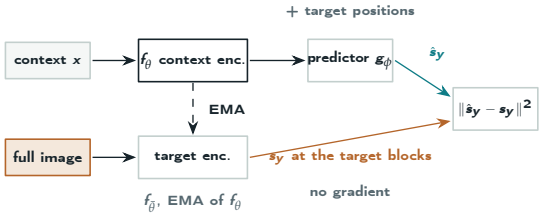


orange: target blocks, the real patches to be predicted; grey: context

- Cut the image into patches, as a ViT does (L10).
- **Targets:** 4 blocks, each covering 15 to 20% of the image.
- **Context:** one large block, 85 to 100%, with the targets cut out.
- Targets large enough to need semantics (a dog's head), context big enough to infer them from.

I-JEPA

2 of 2: two encoders and a predictor (redrawn from Fig. 3)



teal: the predicted codes; orange: target codes computed from the real image

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^{M} \sum_{j \in B_i} \left\| \hat{s}_{y_j} - s_{y_j} \right\|_2^2, \qquad \bar{\theta} \leftarrow \tau\, \bar{\theta} + (1 - \tau)\, \theta$$

The **target encoder** $\bar{\theta}$ is a slow copy of θ (τ from 0.996 to 1) and gets no gradient. Without that, both encoders could give every image the same code and score a perfect loss: **collapse**.

How JEPA is being extended

follow-up papers

Paper	Data	What is hidden and predicted	What changes
I-JEPA (2023)	images	codes of large image blocks	the original: codes, not pixels
MC-JEPA (2023)	video frames	codes, plus optical flow between frames	learns motion and content in one encoder
A-JEPA (2023)	audio	codes of masked spectrogram patches	the same recipe on sound
V-JEPA (2024)	video	codes of space-time tubes	time; features beat pixels on video too
Point-JEPA (2024)	point clouds	codes of hidden groups of points	3D input
V-JEPA 2 (2025)	video, then robot video	next-frame codes given the arm's action	actions: a world model; plans with CEM

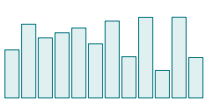
The recipe stays fixed: mask, predict codes, slow target encoder. What changes is the data, and in V-JEPA 2, the actions.

Two families of latent world model

	Reconstruction (PlaNet, Dreamer)	**Latent prediction (JEPA)**
State learned by	decoding frames (ELBO)	predicting the codes of hidden parts
Collapse prevented by	pixel targets are fixed	EMA target, stop-gradient
Actions	in the model from the start	none in I-JEPA; added in V-JEPA 2
Decides by	CEM (PlaNet) or a trained actor (Dreamer)	I-JEPA does not act; V-JEPA 2 plans with CEM
Data	one task's own experience, with reward	unlabelled web images or video
Weak point	capacity spent on irrelevant pixels	no reward; actions only added later

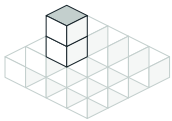
Neither family's state has units or geometry.

Where 3D fits



latent s_t: a vector

no axes, no units



3D state: occupancy, poses

metres, free space, contact

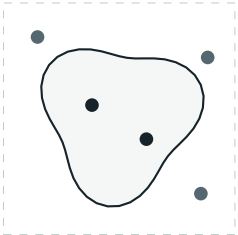
teal: a learned latent state

- None of these models has an explicit 3D state: depth, occlusion and contact are inferred from 2D frames.
- A 3D state gives **units** (metres), **checks** (free space, collision, L07) and a **frame** shared with the actions.
- The cost: the map must be built, and predicted forward too.

The rest of today

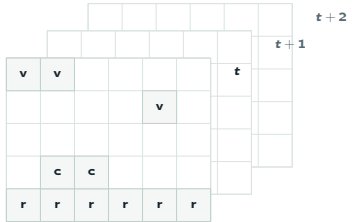
Occupancy forecasting and 4D scene models: predicting the future of a 3D state, a grid in metres.

Two things called occupancy



$o(\mathbf{p}) \in \{0, 1\}$, one object

unit cube, no time, no labels



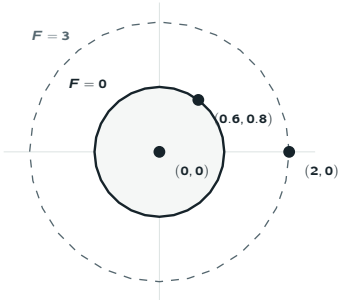
$O_t(x, y, z) \in \{\text{free}, \text{road}, \text{car}, \ldots\}$

a street, labels, and forecasts in time

- **Occupancy Networks** (Mescheder et al., CVPR 2019): $f_\theta(\mathrm{p}) \in [0, 1]$ says whether a point is inside **one object**.
- **Occupancy prediction** in driving (2022 on): a **voxel grid** around the car, each cell free or labelled, later **forecast** seconds ahead.

A shape as a test

an implicit function: the unit circle



$$F(x, y) = x^2 + y^2 - 1$$

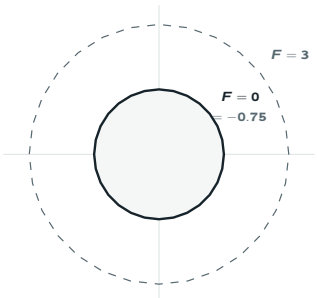
Evaluate F at the three dots. Which are inside?

$F(0,0) = -1$ (inside), $\quad F(0.6, 0.8) = 0$ (on)
$F(2,0) = 3$ (outside)

An **implicit function** stores a shape as a test you can run at any point: negative inside, zero on the boundary, positive outside. Here the boundary $\{F = 0\}$ is the unit circle.

Level sets

contour lines of a function



Which points have $F = c$? Try $c = -0.75$, 0, 3.

$x^2 + y^2 = 1 + c$: circles of radius 0.5, 1, 2.

The **level set** $\{\mathrm{p} : F(\mathrm{p}) = c\}$ is every point with value c. Like contour lines on a hiking map.

ONet picks one level, τ, of a probability, and calls it the surface.

Inside or outside: a score becomes a probability

illustrative scores



$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

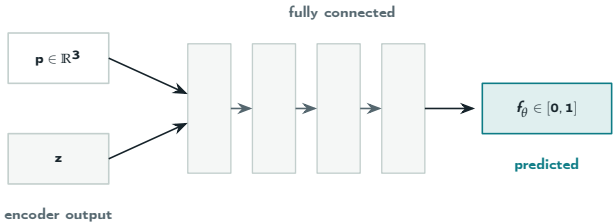
σ at $s = 2,\ 0,\ -1.4$?

0.881, 0.500, 0.198.

A network outputs one number s per point. The **sigmoid** squeezes it into $(0, 1)$: the probability the point is inside.

The occupancy function

1 of 3: a network answers one query

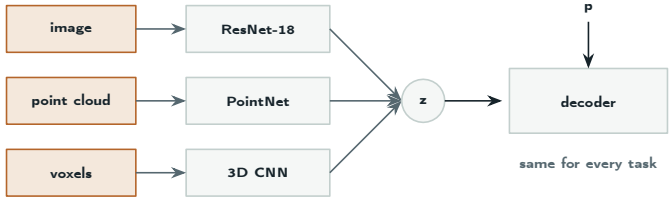


$$f_\theta(\mathrm{p}, \mathrm{z}) \approx \Pr\left(o(\mathrm{p}) = 1 \mid \text{observation}\right)$$

- A probability: a sigmoid (two slides back) on the network's output score.
- z says *which* shape: one network stores a whole family.

The occupancy function

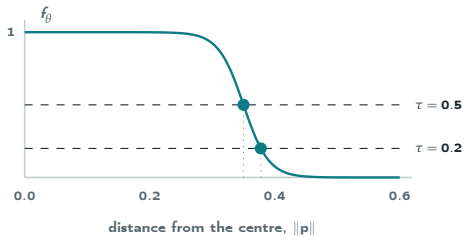
2 of 3: an encoder per input, one decoder



- **Encoder:** turns the observation (one image, a point cloud, or voxels) into a short code z.
- **Decoder:** the same network for every shape. Given z and a point, it answers *inside?*

The surface is a level set

3 of 3: a sphere of radius 0.35 (illustrative network)

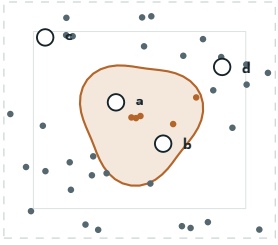


$$f(p) = \sigma\big(50\,(0.35 - \|p\|)\big)$$

$$\text{surface} = \{p : f_\theta(p, z) = \tau\}, \qquad \text{ONet uses } \tau = 0.2$$

- At the centre $f \approx 1$ (inside); at $\|p\| = 0.5$, $f \approx 0.0006$ (outside).
- The surface is where f crosses a threshold τ, the level set from before.
- $\tau = 0.5$: radius 0.350. $\tau = 0.2$: radius 0.378.
- A lower τ moves the surface out by 2.8 cm and keeps thin parts the network is unsure about.

Training: sample points, label them



box, padded

- For each training shape, draw $K = 2{,}048$ points **uniformly** in its padded bounding box.
- Read the true label o of each point from the watertight mesh (orange: observed).
- Fresh points every iteration. Each point is scored with **binary cross-entropy (BCE)**, next slide.

$$\mathcal{L}(\theta, \psi) = \sum_{i=1}^{B} \sum_{j=1}^{K} \mathrm{BCE}\big(f_\theta(\mathrm{p}_{ij}, g_\psi(x_i)),\ o_{ij}\big)$$

The loss by hand

four sampled points (illustrative predictions)

$$\mathrm{BCE}(f, o) = -\big[\, o \ln f + (1 - o) \ln(1 - f) \,\big]$$

point	label o	f_θ	loss
a	inside	0.9	?
b	inside	0.6	?
c	outside	0.2	?
d	outside	0.7	?

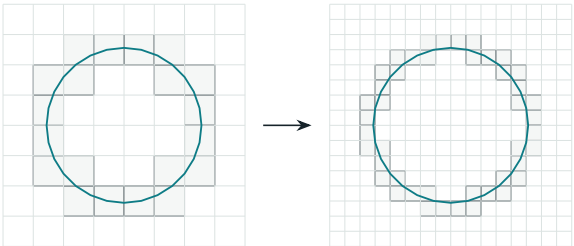
BCE: minus the log of the probability given to the true label. Find each loss, the mean, and $f - o$. Which point is pushed hardest?

$$-\ln 0.9 = 0.105, \quad -\ln 0.6 = 0.511, \quad -\ln(1 - 0.2) = 0.223, \quad -\ln(1 - 0.7) = 1.204$$

$$\text{mean} = 0.511, \qquad f - o = -0.1,\ -0.4,\ +0.2,\ +0.7$$

From a function to a mesh: MISE

evaluate only near the surface



start: 8×8

20 active cells

one step: 16×16

44 active cells

1. Evaluate f_θ on a coarse grid, 32^3.
2. A cell is **active** if its corners disagree about $f_\theta > \tau$.
3. Split each active cell in eight; evaluate only the new corners.
4. Repeat to the target resolution. **Marching cubes** (L06) then turns the in/out values at the corners into triangles.

An octree (L06), grown by asking the network. Like mapping a coastline: zoom in only on squares that hold both land and sea.

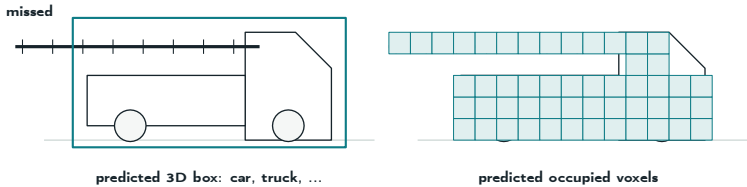
How well does it work?

ONet Table 1: one image to a 3D shape, ShapeNet, mean over classes

Method (what it outputs)	**IoU ↑**
3D-R2N2 (voxels)	0.493
PSGN (points)	n/a
Pix2Mesh (mesh)	0.480
AtlasNet (patches)	n/a
ONet (a function)	0.571

- **IoU** (L11): the overlap of the predicted and the true volume, divided by their union. 1 is perfect.
- Points and patches have no inside, so they get no IoU.

Why driving wanted occupancy



- A detector outputs a box for each class it was trained on. A ladder over a tailgate gets no box, so it is no obstacle.
- Occupancy asks first: **is this space full?** A class label is extra.

The task: a labelled grid from cameras

Occ3D-nuScenes, Tian et al., NeurIPS 2023



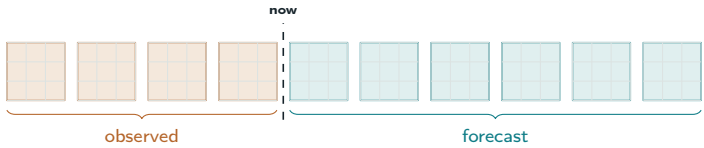
- Six surround cameras in; a grid of $80 \times 80 \times 6.4$ m out, 0.4 m voxels.
- Each voxel: free, or one of 17 classes.

How many voxels per frame? Bytes at one label per byte? Per hour of driving at 2 Hz?

$200 \cdot 200 \cdot 16 = 640{,}000$ voxels, 640 kB a frame, 4.6 GB an hour. 18 labels need only $\log_2 18 = 4.17$ bits each.

From now to next

occupancy forecasting



given $O_{t-T+1}, \ldots, O_t$ (and the ego path), predict $O_{t+1}, \ldots, O_{t+H}$

- Horizons of **1 to 3 s**. A planner checks candidate paths against the forecast without knowing *what* the obstacle is.
- **No future labels:** lidar only says where each ray stopped.

UnO: occupancy as a 4D field

Agro et al., CVPR 2024



$o_\theta(x, y, z, t) \in [0, 1]$, trained with BCE on points labelled from future lidar rays

- A lidar ray says two things: the space **before** the hit is empty, and the spot **at** the hit is full.
- So points before the return are labelled free, points just behind it (within 0.1 m) occupied. No human labels.

The two occupancies meet

	Occupancy Networks	**UnO**
Function	$f_\theta(\mathrm{p}, \mathrm{z})$	$o_\theta(x, y, z, t)$
Extent	one object, unit cube	a street, plus time
Labels	inside test on a watertight mesh	free and occupied points along lidar rays
Sampling	2,048 points, uniform in a box	900,000 each side, along rays
Loss	BCE	BCE

The same recipe in both: sample points, classify each as inside or outside, train with BCE. What changed is where the labels come from.

Learn it again, differently

other explanations of today's material

- Sutton and Barto, Reinforcement Learning: An Introduction, free online: Chapters 3 and 6 cover today's RL slides at a gentler pace.
- The DreamerV3 page, with videos of every domain, and Meta's post on I-JEPA.
- The Occupancy Networks code, with MISE in a few hundred lines, and the authors' CVPR 2019 slides.
- The Occ3D benchmark.

What you should be able to do

- Explain return, value, TD learning and actor-critic, and say where each lives in DreamerV3.
- Say how I-JEPA chooses its masks and why it predicts codes, and what later JEPAs change.
- Say what a level set is, and why ONet's surface is the level set at $\tau = 0.2$.
- Say what Occupancy Networks learn and how they are trained; compute the BCE loss on a few sampled points; explain MISE.
- Say what driving occupancy predicts and what forecasting adds, and how UnO labels points from lidar without human labels.

Next

- **Response papers due Sun Nov 8:** DreamerV3, and Occupancy Networks.
- **Mon Nov 9: Seminar II** (Unit 2) and **Wed Nov 11: Seminar III** (Unit 3), student-led. I am at CoRL.
- **Fri Nov 13:** project checkpoint.
- **L18, Mon Nov 16:** uncertainty, failure, and evaluation.